반도체장비예지보전실무 07

시계열 데이터 다루기

: NASA 터보팬 데이터셋

김 민 진 | muea.mj@gmail.com

시계열 데이터의 전처리

결측치 처리와 데이터 평활화

결측치 처리

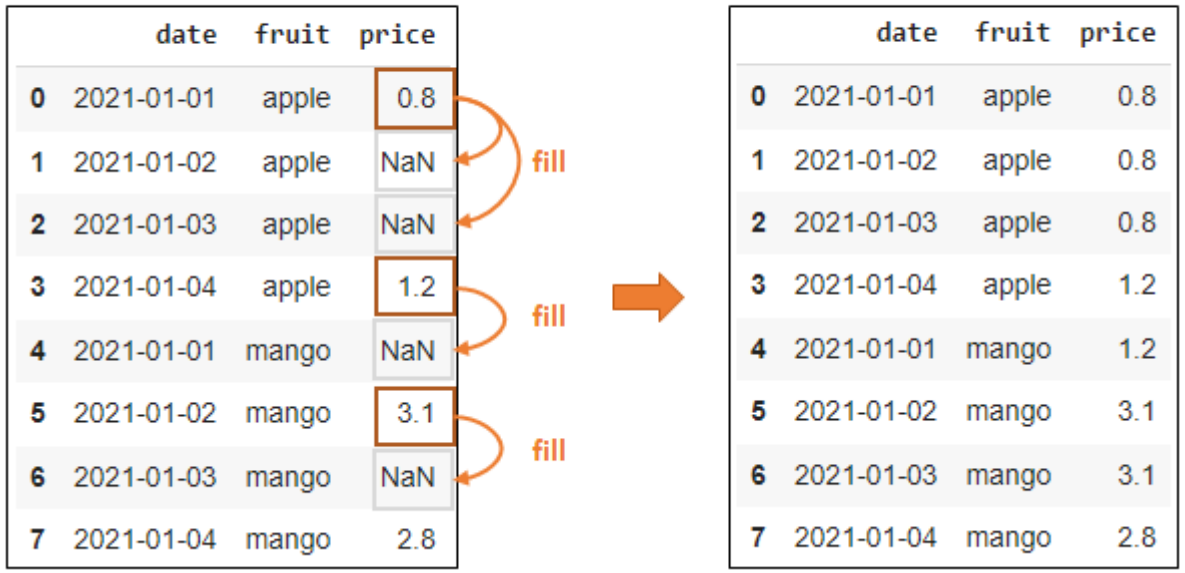
- 시계열 데이터의 결측치 처리 방법

Forward Fill (ffill)	- 이전 시점의 값으로 채움 - 데이터 변화가 크지 않을 때 유용
Backward Fill (bfill)	- 다음 시점의 값으로 채움
선형 보간 (Linear Interpolation)	- 누락된 지점의 양옆 값을 이용해 직선을 그려 중간값을 추정 - 부드러운 변화를 가정할 때 효과적

결측치 처리

- Forward Fill (ffill)

```
df['price'].fillna(method = 'ffill', inplace = True)
```



	date	fruit	price
0	2021-01-01	apple	0.8
1	2021-01-02	apple	NaN
2	2021-01-03	apple	NaN
3	2021-01-04	apple	1.2
4	2021-01-01	mango	NaN
5	2021-01-02	mango	3.1
6	2021-01-03	mango	NaN
7	2021-01-04	mango	2.8

	date	fruit	price
0	2021-01-01	apple	0.8
1	2021-01-02	apple	0.8
2	2021-01-03	apple	0.8
3	2021-01-04	apple	1.2
4	2021-01-01	mango	1.2
5	2021-01-02	mango	3.1
6	2021-01-03	mango	3.1
7	2021-01-04	mango	2.8

→ 최근에는 아래 방식이 권장됨

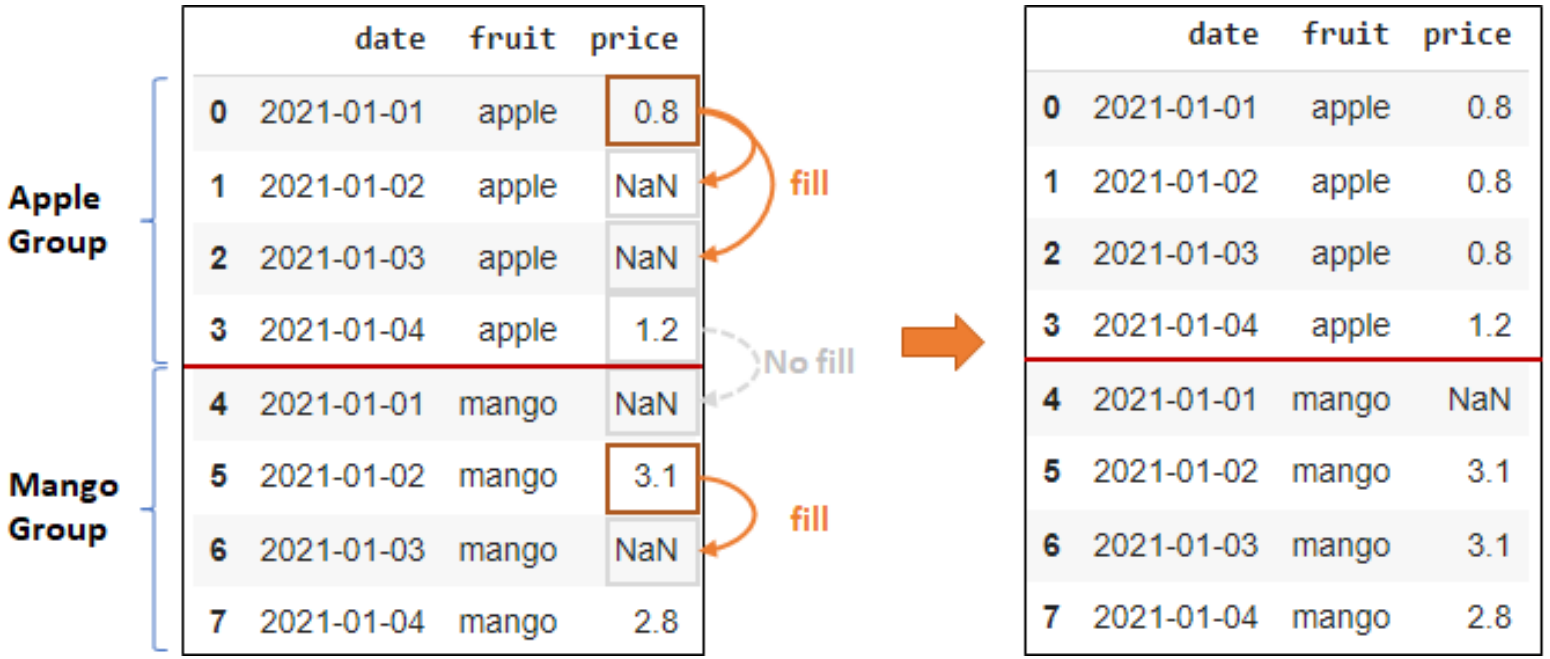
```
df['price'] = df['price'].ffill()
```

https://towardsdatascience.com/how-to-fill-missing-data-with-pandas-8cb875362a0d/

결측치 처리

- **그룹별로** Forward Fill (ffill) 적용하기

```
df['price'] = df.groupby('fruit')['price'].ffill()
```



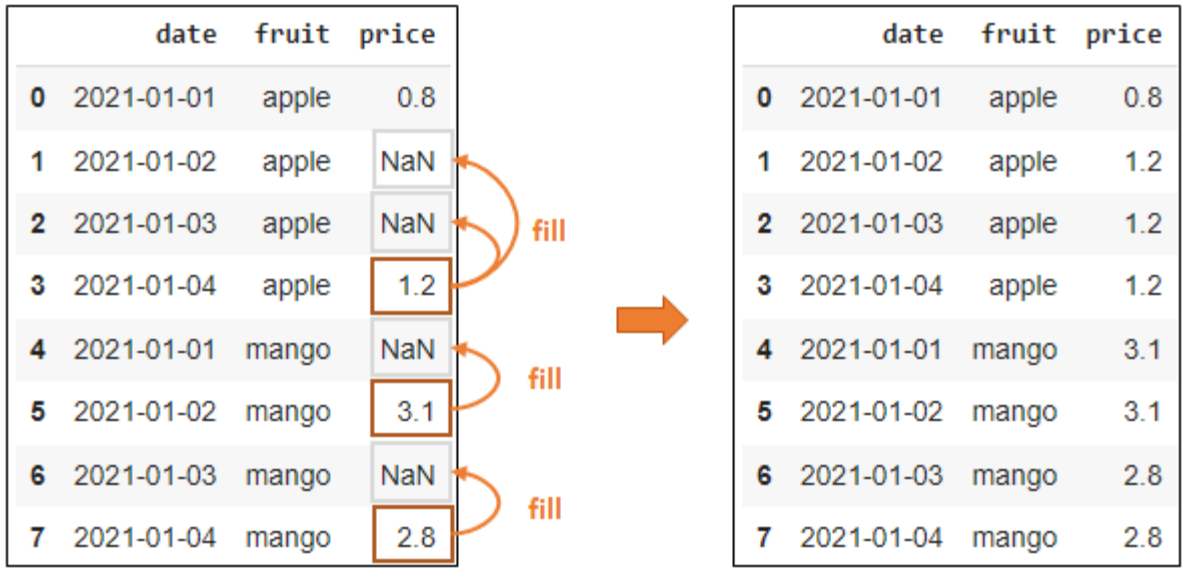
	date	fruit	price
0	2021-01-01	apple	0.8
1	2021-01-02	apple	NaN
2	2021-01-03	apple	NaN
3	2021-01-04	apple	1.2
4	2021-01-01	mango	NaN
5	2021-01-02	mango	3.1
6	2021-01-03	mango	NaN
7	2021-01-04	mango	2.8

	date	fruit	price
0	2021-01-01	apple	0.8
1	2021-01-02	apple	0.8
2	2021-01-03	apple	0.8
3	2021-01-04	apple	1.2
4	2021-01-01	mango	NaN
5	2021-01-02	mango	3.1
6	2021-01-03	mango	3.1
7	2021-01-04	mango	2.8

결측치 처리

- Backward Fill (bfill)

```
df['price'].fillna(method = 'bfill', inplace = True)
```



	date	fruit	price
0	2021-01-01	apple	0.8
1	2021-01-02	apple	NaN
2	2021-01-03	apple	NaN
3	2021-01-04	apple	1.2
4	2021-01-01	mango	NaN
5	2021-01-02	mango	3.1
6	2021-01-03	mango	NaN
7	2021-01-04	mango	2.8

	date	fruit	price
0	2021-01-01	apple	0.8
1	2021-01-02	apple	1.2
2	2021-01-03	apple	1.2
3	2021-01-04	apple	1.2
4	2021-01-01	mango	3.1
5	2021-01-02	mango	3.1
6	2021-01-03	mango	2.8
7	2021-01-04	mango	2.8

→ 최근에는 아래 방식이 권장됨

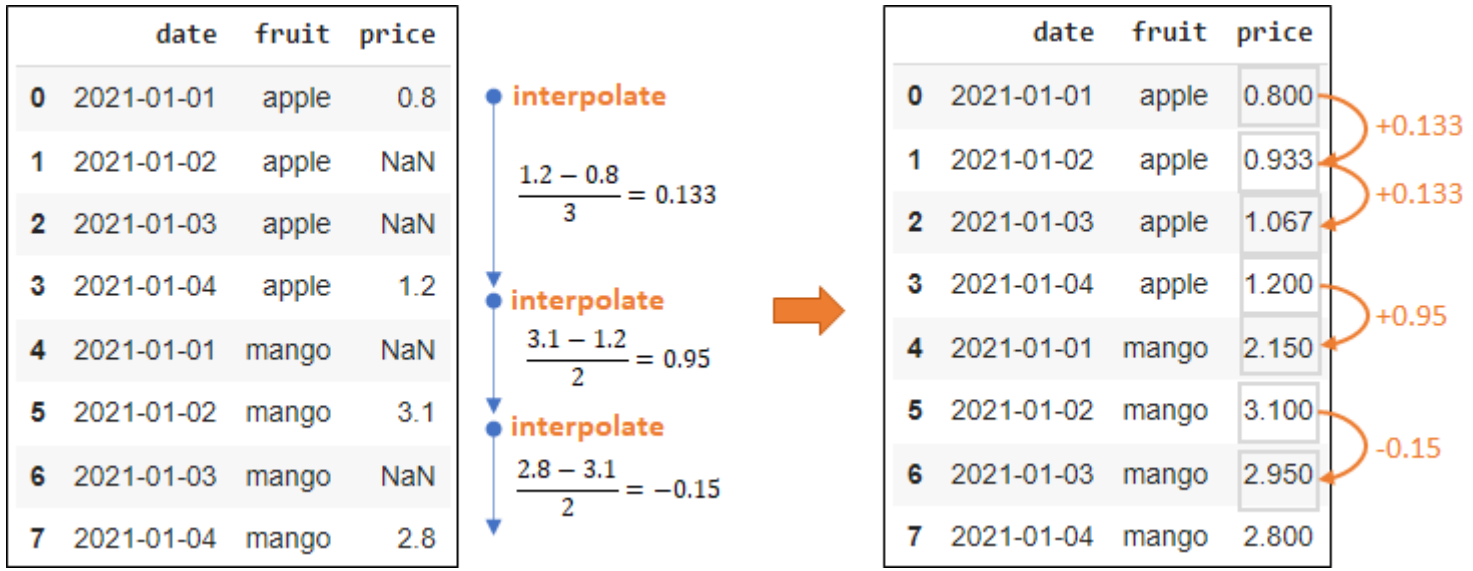
```
df['price'] = df['price'].bfill()
```

https://towardsdatascience.com/how-to-fill-missing-data-with-pandas-8cb875362a0d/

결측치 처리

- 선형 보간 (Linear Interpolation)

```
df['price'].interpolate(method = 'linear', inplace = True)
```

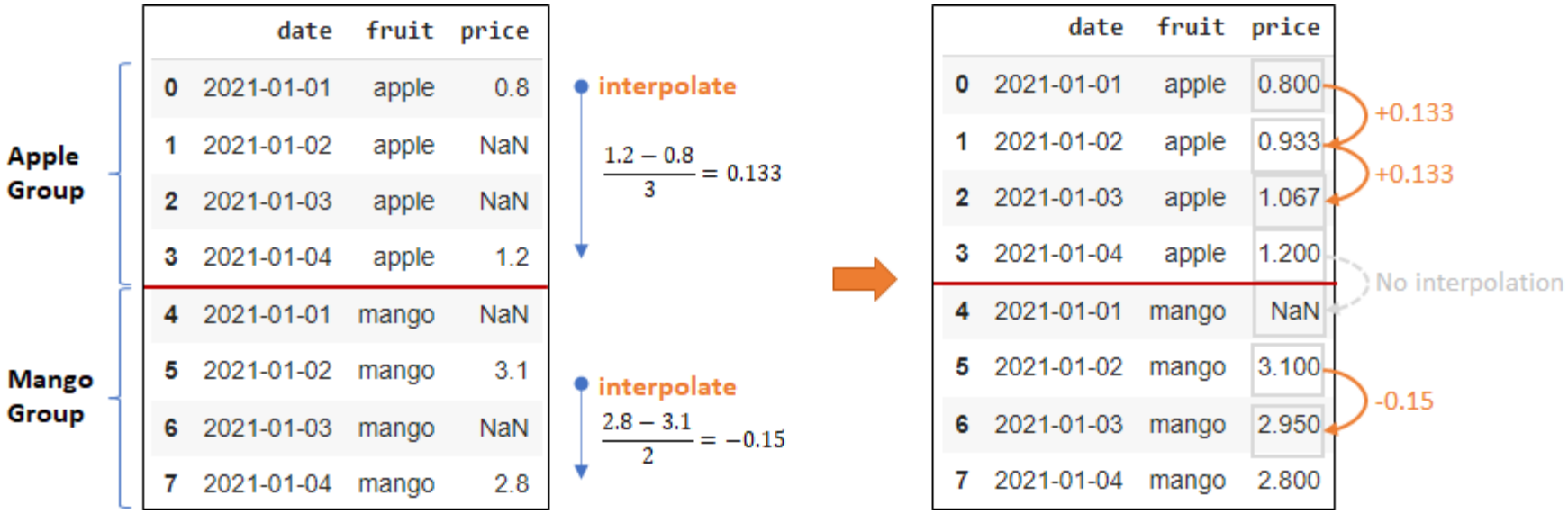


https://towardsdatascience.com/how-to-fill-missing-data-with-pandas-8cb875362a0d/

결측치 처리

- **그룹별로** 선형 보간 (Linear Interpolation)

```
df.groupby('fruit')['price'].apply(lambda x: x.interpolate(method='linear'))
```



	date	fruit	price
0	2021-01-01	apple	0.8
1	2021-01-02	apple	NaN
2	2021-01-03	apple	NaN
3	2021-01-04	apple	1.2
4	2021-01-01	mango	NaN
5	2021-01-02	mango	3.1
6	2021-01-03	mango	NaN
7	2021-01-04	mango	2.8

	date	fruit	price
0	2021-01-01	apple	0.800
1	2021-01-02	apple	0.933
2	2021-01-03	apple	1.067
3	2021-01-04	apple	1.200
4	2021-01-01	mango	NaN
5	2021-01-02	mango	3.100
6	2021-01-03	mango	2.950
7	2021-01-04	mango	2.800

https://towardsdatascience.com/how-to-fill-missing-data-with-pandas-8cb875362a0d/

실습 데이터: NASA 터보엔진 데이터셋

- 데이터셋 다운로드
  - https://www.Kaggle.com/datasets/behrad3d/nasa-cmaps

- 데이터셋 구성
  - 4개의 서브셋(FD001, FD002, FD003, FD004)으로 구성
    - train_FD001.txt: 훈련 데이터 (100개의 엔진)
    - test_FD001.txt: 테스트 데이터 (100개의 엔진)
    - RUL_FD001.txt: 테스트 데이터의 실제 RUL 값

- 데이터 구성
  - **열 1**: 엔진 ID
  - **열 2**: 사이클(시간)
  - **열 3 ~ 5**: 운영 설정 1~3
  - **열 6 ~ 26**: 21개의 센서 데이터

결측치 처리 실습

- 데이터 로딩

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. 데이터 불러오기
columns_name = ['unit_number', 'cycles', 'op_setting_1', 'op_setting_2', 'op_setting_3'] + \
               [ f'sensor_{i}' for i in range(1, 22) ]

df = pd.read_csv('/content/drive/MyDrive/_PdM/NASA/train_FD001.txt', sep=r'\s+', header=None, names=columns_name)
```

```
# 2. 1번 엔진의 'sensor_2' 데이터만 따로 복사하여 실습 진행
engine_1_sensor_2 = df[df['unit_number'] == 1][['cycles', 'sensor_2']].copy()
```

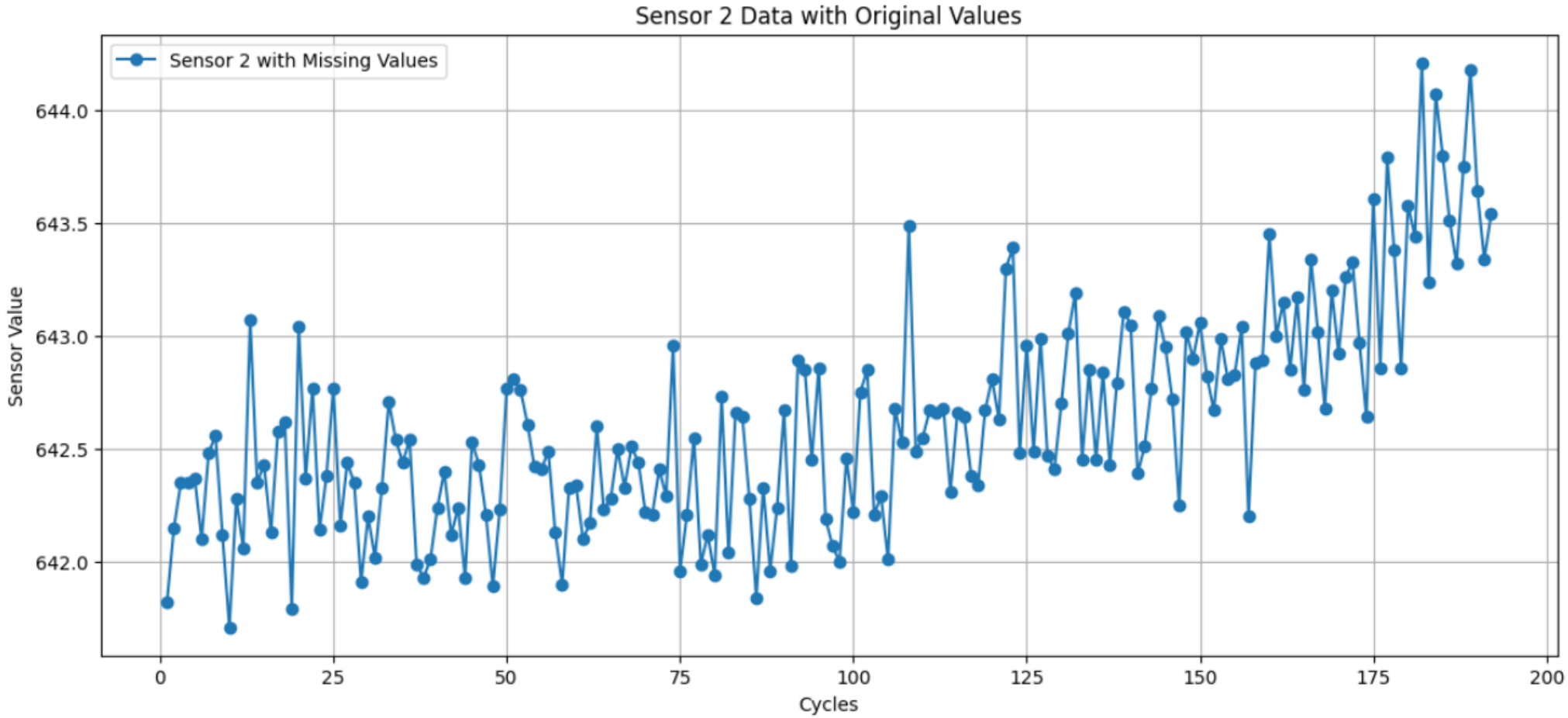
결측치 처리 실습

- 원본 데이터 확인

```
# 원본 데이터 확인 (시각화)
plt.figure(figsize=(14, 6))
plt.plot(engine_1_sensor_2['cycles'], engine_1_sensor_2['sensor_2'], 'o-', label='Sensor 2 with Missing Values')
plt.title('Sensor 2 Data with Original Values')
plt.xlabel('Cycles')
plt.ylabel('Sensor Value')
plt.legend()
plt.grid(True)
plt.show()
```

결측치 처리 실습

- 원본 데이터 확인



결측치 처리 실습

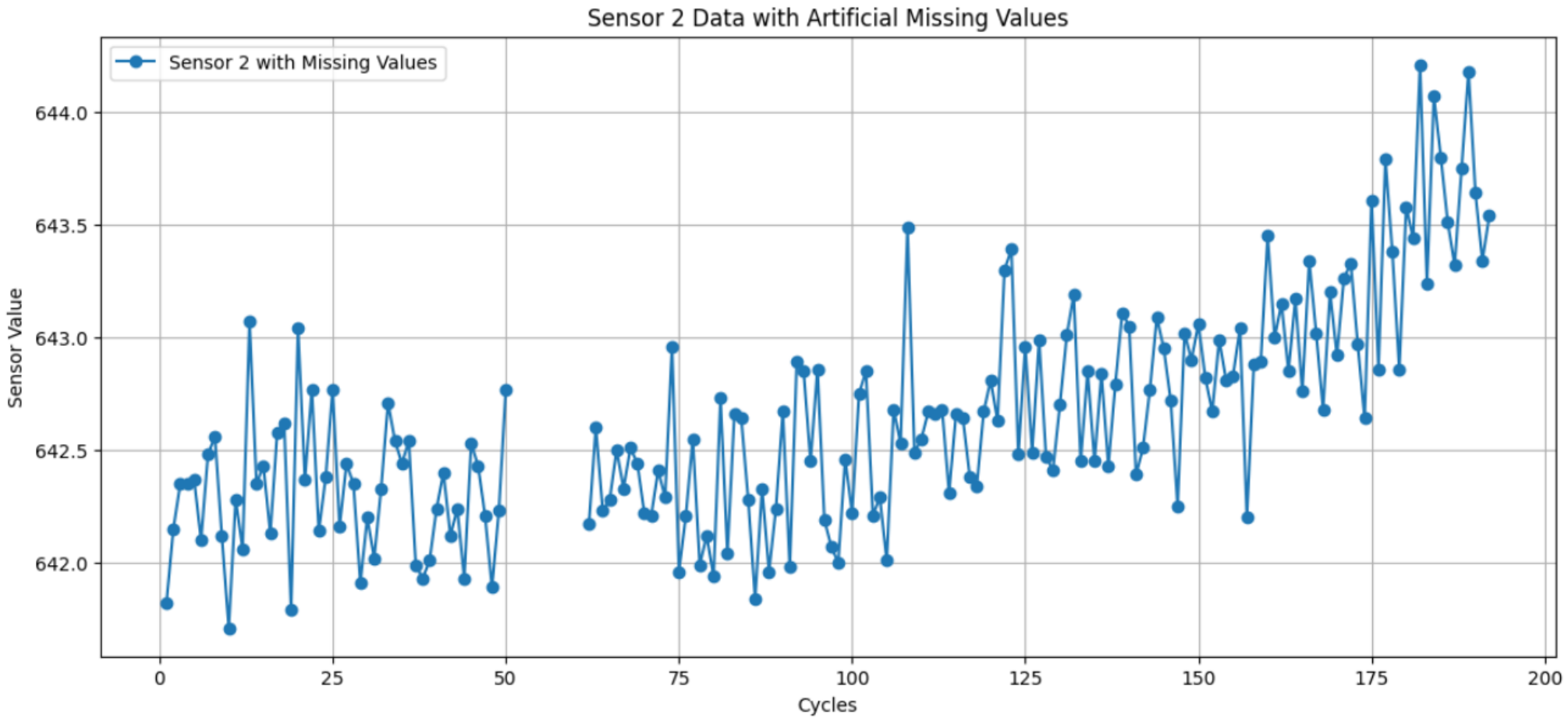
- 인위적으로 결측치 만들기

```
# 3. 인위적으로 결측치 만들기 (예: 50~60 사이클 구간의 데이터를 삭제)
engine_1_sensor_2.loc[50:60, 'sensor_2'] = np.nan

# 4. 결측치 확인 (시각화)
plt.figure(figsize=(14, 6))
plt.plot(engine_1_sensor_2['cycles'], engine_1_sensor_2['sensor_2'], 'o-', label='Sensor 2 with Missing Values')
plt.title('Sensor 2 Data with Artificial Missing Values')
plt.xlabel('Cycles')
plt.ylabel('Sensor Value')
plt.legend()
plt.grid(True)
plt.show()
```

결측치 처리 실습

- 인위적으로 결측치 만든 결측치 확인



결측치 처리 실습

- 결측치 처리
  - Forward Fill (ffill) : 이전 값으로 채우기
  - 선형 보간 (linear interpolate) : 누락된 지점의 양옆 값을 이용해 직선을 그려 중간값을 추정

```
# --- 이제 결측치를 채워봅니다 ---

# 5-1. Forward Fill 적용
engine_1_sensor_2['sensor_2_ffill'] = engine_1_sensor_2['sensor_2'].ffill()

# 5-2. 선형 보간 적용
engine_1_sensor_2['sensor_2_interpolated'] = engine_1_sensor_2['sensor_2'].interpolate(method='linear')
```

결측치 처리 실습

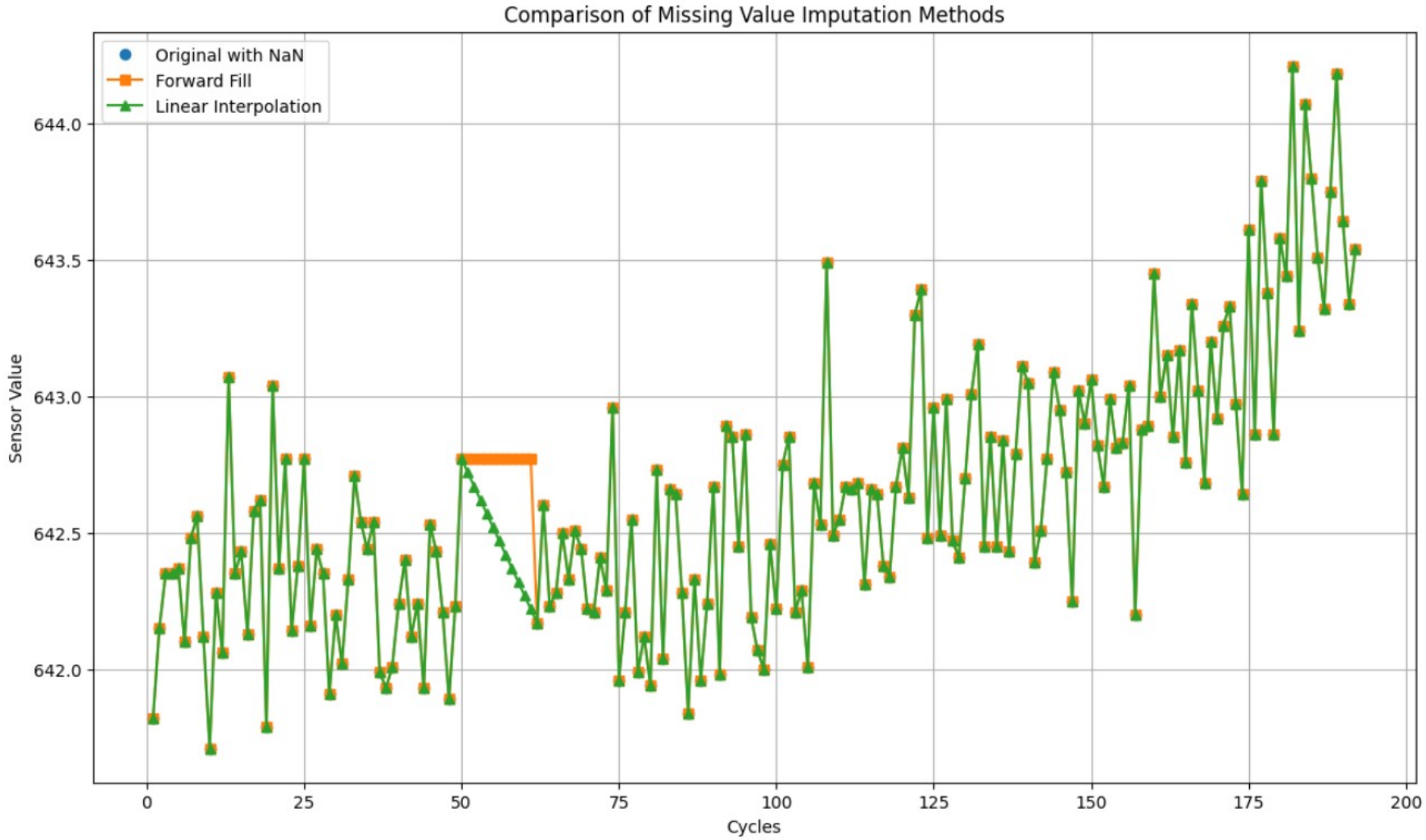
- 결측치 처리 결과 확인

```
# 6. 결과 비교 시각화
plt.figure(figsize=(14, 8))
# 원본 + 결측치
plt.plot(engine_1_sensor_2['cycles'], engine_1_sensor_2['sensor_2'], 'o', label='Original with NaN')
# ffill 결과
plt.plot(engine_1_sensor_2['cycles'], engine_1_sensor_2['sensor_2_ffill'], 's-', label='Forward Fill')
# 선형 보간 결과
plt.plot(engine_1_sensor_2['cycles'], engine_1_sensor_2['sensor_2_interpolated'], '^-', label='Linear Interpolation')

plt.title('Comparison of Missing Value Imputation Methods')
plt.xlabel('Cycles')
plt.ylabel('Sensor Value')
plt.legend()
plt.grid(True)
plt.show()
```

결측치 처리 실습

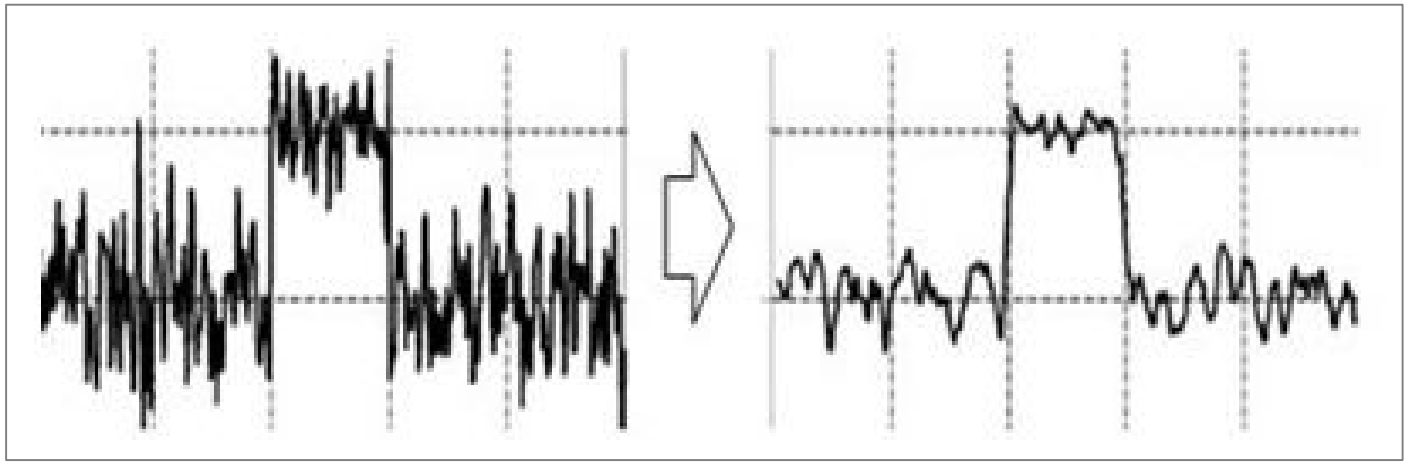
- 결측치 처리 결과 확인



노이즈 제거

- 필요성
  - 센서 데이터에는 장비의 실제 상태 변화와 상관없는 미세한 노이즈(Noise)가 섞여 있는 경우가 많음
  - 이러한 노이즈는 모델의 학습을 방해할 수 있음
- 이동 평균법(Moving Average)
  - 단기적인 변동(노이즈)을 완화시켜 데이터의 장기적인 추세를 파악하는 전처리 기법



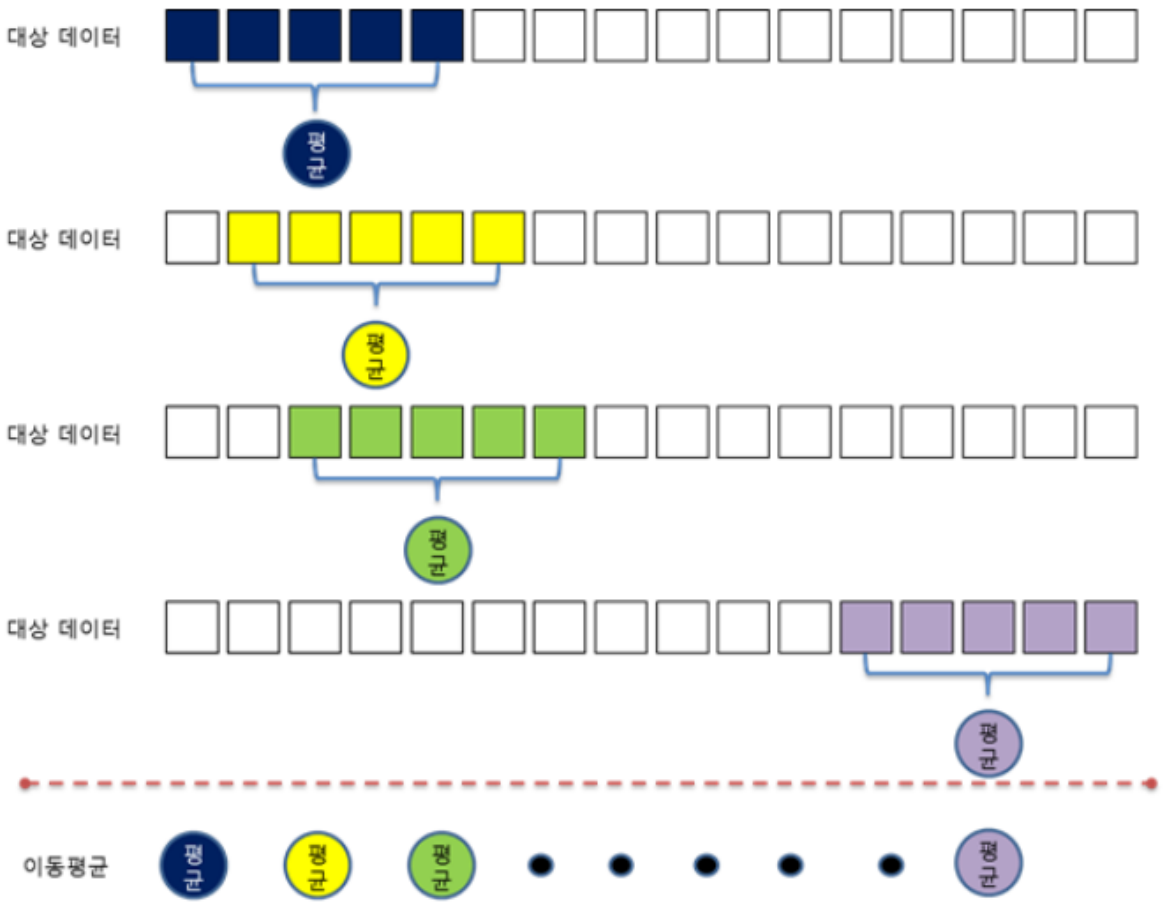


https://namu.wiki/w/%EC%9D%B4%EB%8F%99%ED%8F%89%EA%B7%A0%EC%84%A0 http://www.ktword.co.kr/test/view/view.php?no=3665

노이즈 제거

- 이동 평균법의 원리
  - "움직이는 창문(Sliding Window)"을 데이터 위로 이동시킴
  - 창문 안에 있는 데이터들의 평균을 계산하여 현재 값을 대체

- 중요한 것은 Window Size!



https://m.blog.naver.com/ntkor/199891958

노이즈 제거 실습 : rolling 함수 활용

- Window 크기에 따라 결과 그래프가 어떻게 변하는지 확인해보자

```
# 1. 1번 엔진의 'sensor_7' 데이터 사용 (변동성이 비교적 큰 센서)
engine_1_sensor_7 = df[df['unit_number'] == 1][['cycles', 'sensor_7']].copy()

# 2. 다양한 윈도우 크기로 이동 평균 계산
window_size_small = 5
window_size_large = 20

engine_1_sensor_7['rolling_mean_small'] = engine_1_sensor_7['sensor_7'].rolling(window=window_size_small).mean()
engine_1_sensor_7['rolling_mean_large'] = engine_1_sensor_7['sensor_7'].rolling(window=window_size_large).mean()
```

노이즈 제거 실습 : rolling 함수 활용

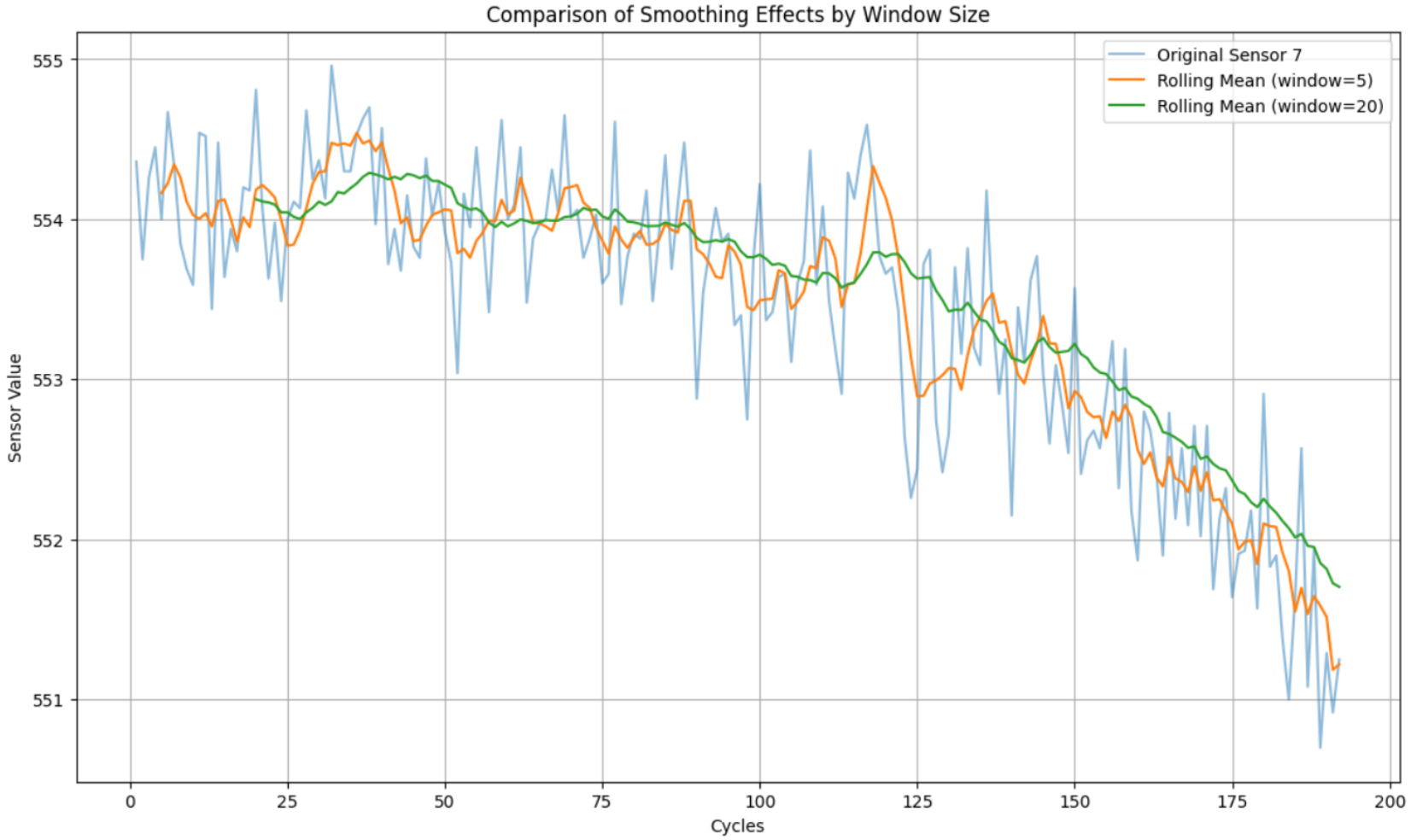
- Window 크기에 따라 결과 그래프가 어떻게 변하는지 확인해보자

```
# 3. 결과 비교 시각화
plt.figure(figsize=(14, 8))
# 원본 데이터
plt.plot(engine_1_sensor_7['cycles'], engine_1_sensor_7['sensor_7'], label='Original Sensor 7', alpha=0.5)
# 작은 윈도우 이동 평균
plt.plot(engine_1_sensor_7['cycles'], engine_1_sensor_7['rolling_mean_small'], label=f'Rolling Mean (window={window_size_small})')
# 큰 윈도우 이동 평균
plt.plot(engine_1_sensor_7['cycles'], engine_1_sensor_7['rolling_mean_large'], label=f'Rolling Mean (window={window_size_large})')

plt.title('Comparison of Smoothing Effects by Window Size')
plt.xlabel('Cycles')
plt.ylabel('Sensor Value')
plt.legend()
plt.grid(True)
plt.show()
```

노이즈 제거 실습 : rolling 함수 활용

- Window 크기에 따라 결과 그래프가 어떻게 변하는지 확인해보자



노이즈 제거 실습 고찰

- 어떤 Window 크기가 최적인가?
  - 정답은 없으며, 분석의 목적과 데이터의 특성에 따라 달라짐
  - **Window가 작으면**
    - 노이즈 제거 효과가 적지만, 데이터의 급격한 변화를 잘 포착
  - **Window가 크면**
    - 노이즈 제거 효과가 크고(매우 부드러운 선), 데이터의 세부적인 변화는 놓칠 수 있음
  - **적용 포인트:** 미세한 이상 징후를 잡으려면 작은 윈도우, 전반적인 트렌드를 보려면 큰 윈도우

시계열 데이터의 특성공학

롤링 통계와 지연 특징

롤링 통계 : 데이터의 흐름에서 패턴 찾기

- 개념
  - 시간에 따라 변화하는 데이터의 노이즈를 줄이고, 숨겨진 추세를 파악하기 위한 기법
  - 시간의 흐름에 따른 데이터의 국소적인(Local) 특성 변화를 파악할 수 있음

- 원리
  ① 움직이는 창문(Sliding Window)을 설정
  ② 창문(Window) 안에 들어온 데이터들의 통계치를 계산
  ③ 새로운 특성으로 만듦

- 예지보전에서의 의미
  - 반도체 장비의 마모 과정은 시간에 따른 점진적인 변화로 나타남
  - 이 변화의 '흐름'을 포착하는 것이 예지 보전의 핵심임

롤링 통계 : 데이터의 흐름에서 패턴 찾기

- 주요 통계량

이동 평균 (Rolling Mean)	- 윈도우 내 데이터의 평균 - 단기적인 변동을 제거하고 데이터의 장기적인 추세를 부드럽게 보여줌 - "센서 값이 전반적으로 상승하고 있는가?"
이동 표준편차 (Rolling Standard Deviation)	- 윈도우 내 데이터의 변동성 - 데이터의 변동성 또는 안정성을 보여줌 - 이 값이 갑자기 커진다면 장비가 불안정한 상태임을 의미할 수 있음 - "장비가 고장 나기 직전에 센서 값이 불안정하게 흔들리지 않는가?"
이동 최댓값/최솟값 (Rolling Max/Min)	- 윈도우 내 극단값을 포착함

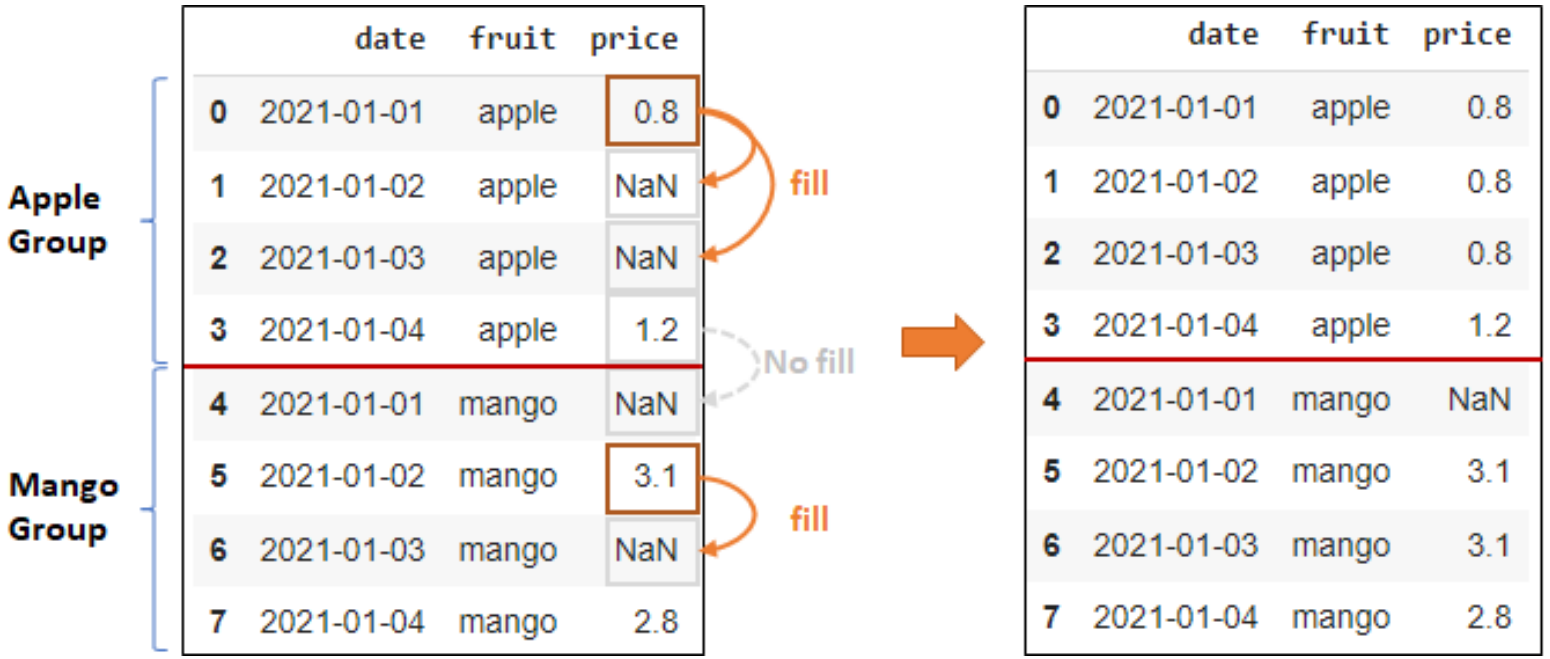
목적에 따라 달리 사용되는 이동평균 이해하기

- 데이터를 깨끗하게 만들 것인가? → **전처리**
- 모델에게 더 많은 정보를 줄 것인가? → **특성 공학**

구분	전처리 (Preprocessing)	특성 공학 (Feature Engineering)
목표	데이터 **정제** 및 노이즈 제거	모델을 위한 **정보(힌트) 추가**
결과물	부드러워진 **단일 시계열**	원본 + **새로운 특성(열)**
데이터 처리	원본 데이터를 **대체**	원본 데이터 옆에 **추가**
핵심 질문	"데이터의 진짜 추세는 무엇인가?"	"이 데이터의 단기적 추세는 예측에 도움이 되는가?"

롤링 통계 : 그룹별로 rolling() 함수 적용하기

```
df['price'] = df.groupby('fruit')['price'].ffill()
```



	date	fruit	price
0	2021-01-01	apple	0.8
1	2021-01-02	apple	NaN
2	2021-01-03	apple	NaN
3	2021-01-04	apple	1.2
4	2021-01-01	mango	NaN
5	2021-01-02	mango	3.1
6	2021-01-03	mango	NaN
7	2021-01-04	mango	2.8

	date	fruit	price
0	2021-01-01	apple	0.8
1	2021-01-02	apple	0.8
2	2021-01-03	apple	0.8
3	2021-01-04	apple	1.2
4	2021-01-01	mango	NaN
5	2021-01-02	mango	3.1
6	2021-01-03	mango	3.1
7	2021-01-04	mango	2.8

롤링 통계 실습: Pandas rolling() 함수 활용

- 데이터 로딩

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. 데이터 불러오기
columns_name = ['unit_number', 'cycles', 'op_setting_1', 'op_setting_2', 'op_setting_3'] + \
               [ f'sensor_{i}' for i in range(1, 22) ]
df = pd.read_csv('/content/drive/MyDrive/_PdM/NASA/train_FD001.txt', sep=r'\s+', header=None, names=columns_name)
```

롤링 통계 실습: Pandas rolling() 함수 활용

- 롤링 통계 계산

```
# 2. 롤링 통계를 계산할 윈도우 크기 설정
window_size = 10

# 3. 이동 평균과 이동 표준편차 특징 추가
# 중요: groupby('unit_number')로 엔진별로 따로 계산해야 함!
df['sensor_3_roll_mean'] = df.groupby('unit_number')['sensor_3'].transform(
    lambda x: x.rolling(window_size, min_periods=1).mean()
)
df['sensor_3_roll_std'] = df.groupby('unit_number')['sensor_3'].transform(
    lambda x: x.rolling(window_size, min_periods=1).std()
)
```

transform의 역할

- 그룹별로 계산한 결과를, 원래 데이터프레임의 '모든 행'에 그대로 다시 붙여줌
- 그룹별 계산을 하지만 결과의 행 개수를 원본과 동일하게 유지시켜줌
- 이 덕분에 새로운 특성(열)으로 바로 추가할 수 있음

롤링 통계 실습: Pandas rolling() 함수 활용

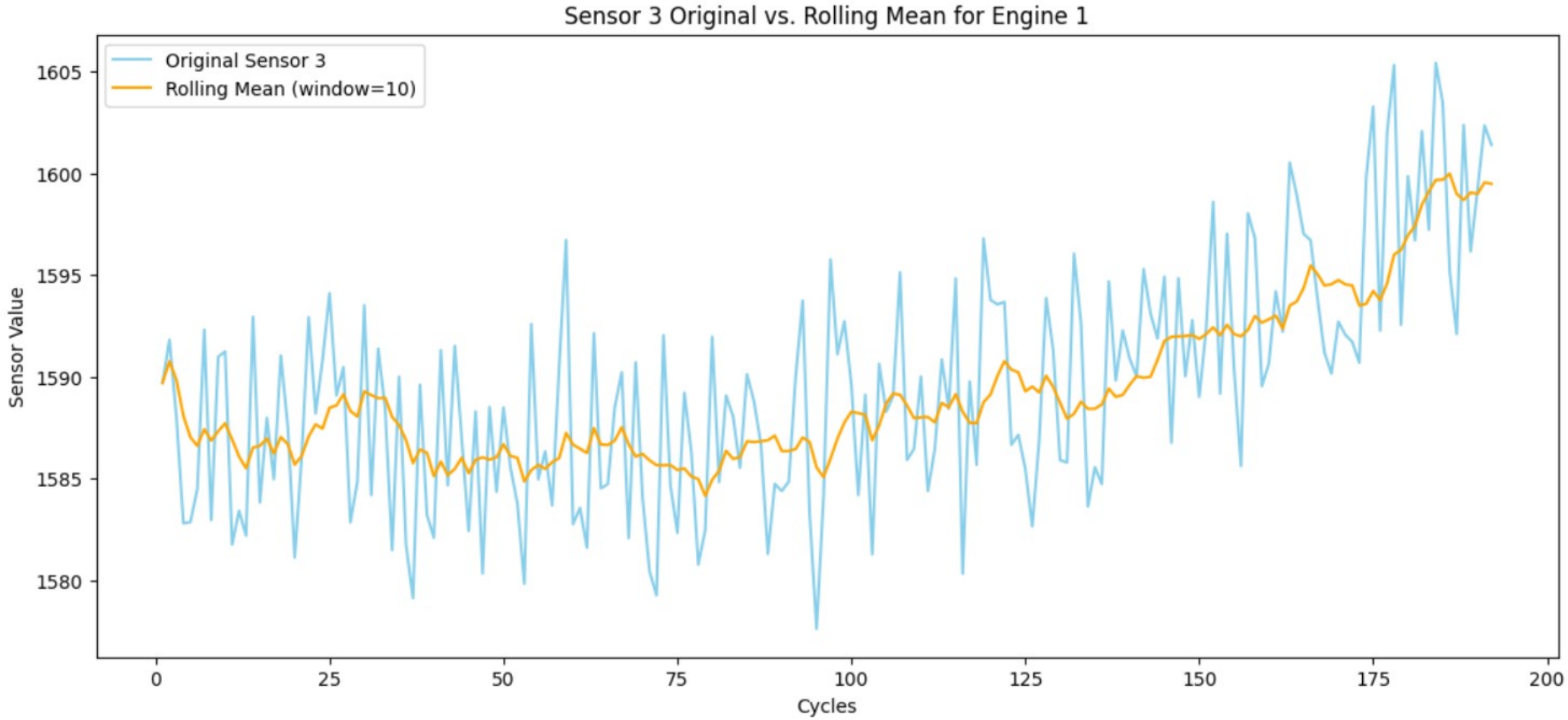
- 결과 시각화: 이동 평균

```
# 4. 특정 엔진(예: 1번 엔진)의 데이터 시각화로 결과 확인
engine_1_df = df[df['unit_number'] == 1]

plt.figure(figsize=(14, 6))
plt.plot(engine_1_df['cycles'], engine_1_df['sensor_3'], label='Original Sensor 3', color='skyblue')
plt.plot(engine_1_df['cycles'], engine_1_df['sensor_3_roll_mean'],
         label=f'Rolling Mean (window={window_size})', color='orange')
plt.legend()
plt.title('Sensor 3 Original vs. Rolling Mean for Engine 1')
plt.xlabel('Cycles')
plt.ylabel('Sensor Value')
plt.show()
```

롤링 통계 실습: Pandas rolling() 함수 활용

- 결과 시각화: 이동 평균



롤링 통계 실습: Pandas rolling() 함수 활용

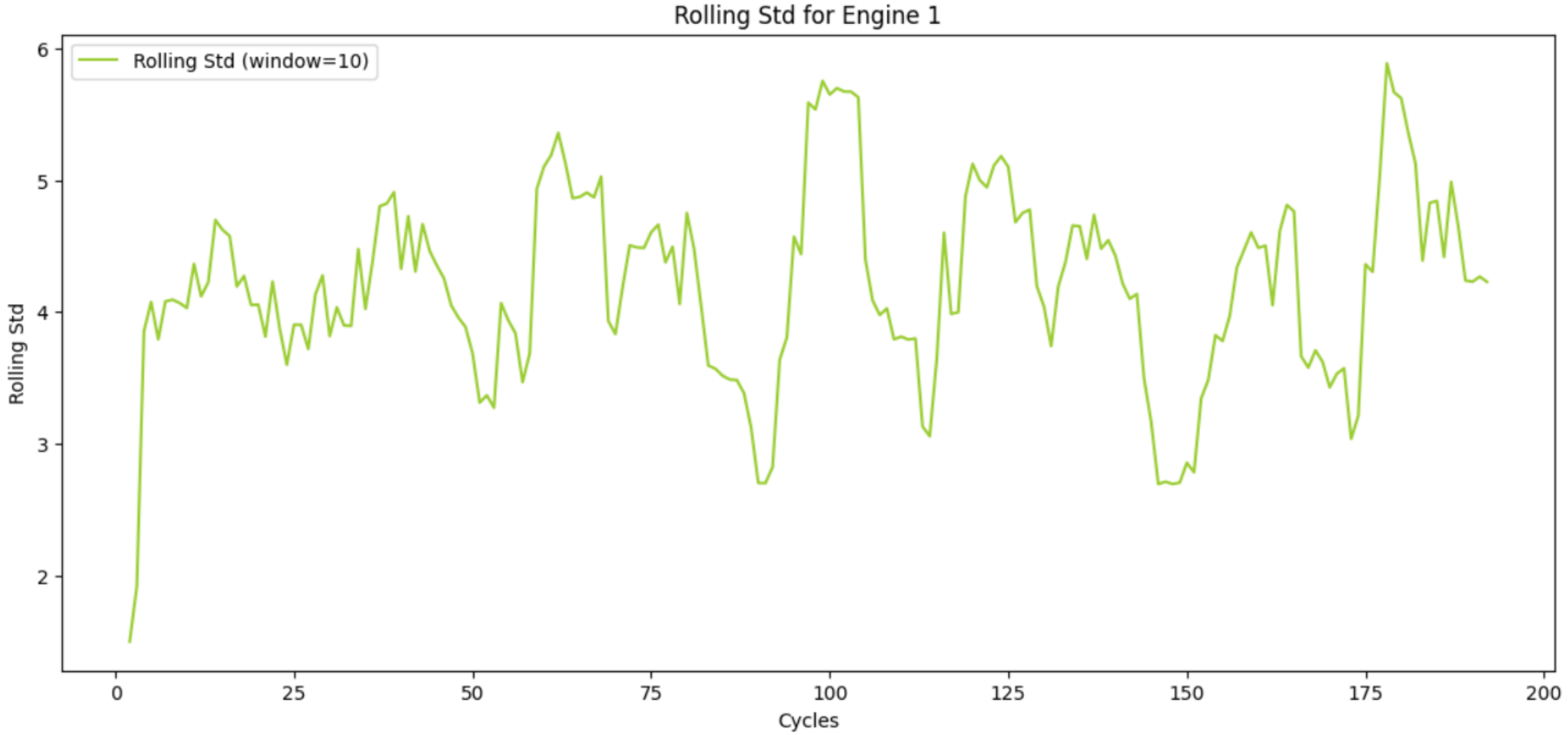
- 결과 시각화 : 이동 편차

```
# 4. 특정 엔진(예: 1번 엔진)의 데이터 시각화로 결과 확인
engine_1_df = df[df['unit_number'] == 1]

plt.figure(figsize=(14, 6))
plt.plot(engine_1_df['cycles'], engine_1_df['sensor_3_roll_std'],
         label=f'Rolling Std (window={window_size})', color='yellowgreen')
plt.legend()
plt.title('Rolling Std for Engine 1')
plt.xlabel('Cycles')
plt.ylabel('Rolling Std')
plt.show()
```

롤링 통계 실습: Pandas rolling() 함수 활용

- 결과 시각화 : 이동 편차



롤링 통계 실습: Pandas rolling() 함수 활용

- 결과 시각화 : 이동 평균과 이동 편차를 한 번에 그리기

```
# 4. 특정 엔진(예: 1번 엔진)의 데이터 시각화로 결과 확인
engine_1_df = df[df['unit_number'] == 1]

plt.rcParams['figure.figsize'] = (14, 6)

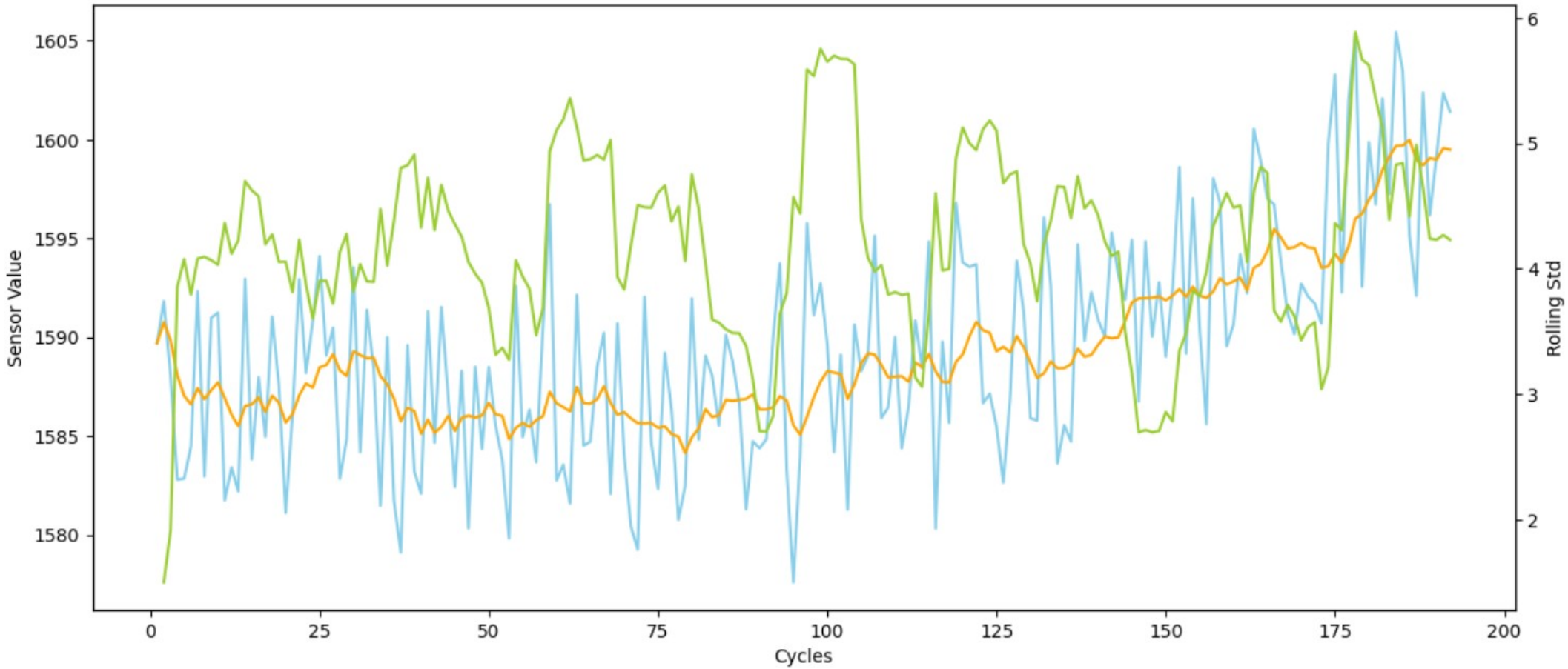
fig, ax1 = plt.subplots()
ax1.plot(engine_1_df['cycles'], engine_1_df['sensor_3'], label='Original Sensor 3', color='skyblue')
ax1.plot(engine_1_df['cycles'], engine_1_df['sensor_3_roll_mean'],
         label=f'Rolling Mean (window={window_size})', color='orange')
ax1.set_xlabel('Cycles')
ax1.set_ylabel('Sensor Value')

ax2 = ax1.twinx()
ax2.plot(engine_1_df['cycles'], engine_1_df['sensor_3_roll_std'],
         label=f'Rolling Std (window={window_size})', color='yellowgreen') #linestyle='--'
ax2.set_ylabel('Rolling Std')

plt.show()
```

롤링 통계 실습: Pandas rolling() 함수 활용

- 결과 시각화 : 이동 평균과 이동 편차를 한 번에 그리기



지연 특성과 누적 통계 : 과거와 현재를 연결하기

- 개념
  - 현재 시점의 데이터가 과거의 어떤 값에 영향을 받았는지 모델에게 알려주기 위해 생성하는 특성

- 지연 특징 (Lag Features)
  - 과거의 데이터를 현재로 가져옴
  - 1주기 전, 2주기 전의 센서 값을 현재 시점의 새로운 특징으로 추가

  - 왜 필요한가?
    - 머신러닝 모델은 기본적으로 각 데이터 행(row)을 독립적으로 봄
    - 지연 특징을 추가함으로써, 모델이 시간의 연속성("1주기 전 온도가 이랬기 때문에 현재 압력이 이렇다")을 학습하게 만들 수 있음

지연 특성과 누적 통계 : 과거와 현재를 연결하기

- 누적 통계 (Cumulative Statistics)
  - 데이터가 시작된 시점부터 현재까지의 통계를 누적하여 계산

  - 누적 합 (Cumulative Sum)
    - 장비가 가동된 이후 받은 총 스트레스 또는 총 작업량을 나타내는 지표로 활용될 수 있습니다.

  - 누적 평균 (Cumulative Mean)
    - 전체 기간 동안의 평균적인 상태를 나타냅니다.

실습: shift()와 cumsum() 함수 활용

- shift() 함수로 바로 이전 사이클의 센서 값을 가져오고, cumsum()으로 센서 값의 누적 합을 계산

```
# 1. 지연 특징 추가 (1 사이클 전 sensor_4 값)
# 중요: 역시 groupby()를 사용하여 엔진별로 계산해야 데이터가 섞이지 않음
df['sensor_4_lag_1'] = df.groupby('unit_number')['sensor_4'].transform(lambda x: x.shift(1))

# 2. 누적 합 특징 추가
df['sensor_7_cumsum'] = df.groupby('unit_number')['sensor_7'].transform(lambda x: x.cumsum())
```

실습: shift()와 cumsum() 함수 활용

```
# 3. 결과 확인 (결측치가 발생하는 첫 행 처리의 중요성도 함께 설명)
# shift()를 사용하면 첫 번째 행에 결측치(NaN)가 생기므로 처리 필요
print("Lag feature 생성 후 결측치 확인:")
df[['unit_number', 'cycles', 'sensor_4', 'sensor_4_lag_1']].head()
```

Lag feature 생성 후 결측치 확인:

	unit_number	cycles	sensor_4	sensor_4_lag_1
0	1	1	1400.60	NaN
1	1	2	1403.14	1400.60
2	1	3	1404.20	1403.14
3	1	4	1401.87	1404.20
4	1	5	1406.22	1401.87

```
# 결측치 채우기 (예: 0으로 채우거나, ffill)
df.fillna(0, inplace=True)
```

	unit_number	cycles	sensor_4	sensor_4_lag_1
0	1	1	1400.60	0.00
1	1	2	1403.14	1400.60
2	1	3	1404.20	1403.14
3	1	4	1401.87	1404.20
4	1	5	1406.22	1401.87

참고 자료

- 『제조AI빅데이터 분석 기법 (코딩편)』, 김일중 외 6인 공저, 보민출판사 (2024)
- 『혼자 공부하는 머신러닝 딥러닝』

끝!